AIQA — Phase 1–6 Final Consolidated Audit

Date: 2026-08-26 **Branch:** `claude/run-this-9eeb2e` (commit `1670eed`, pushed) **Auditor stance:** Evidence-first. Every claim below is backed by a file:line citation, a test-run result, or a live runtime transcript captured during this audit. Nothing is carried forward from a prior report without independent re-verification in this session.

---

1. Executive Summary

The AI-generation -> execution -> evidence chain that Phases 1, 2, 4, and 6 together form is **genuinely real and connected end-to-end** — re-verified in this audit by code inspection, by an independently re-run test suite (298/298 backend, 401/401 frontend, both from a genuinely clean environment), and by live runtime evidence including a real local Ollama call that succeeded, one that honestly timed out, and one that honestly reported a malformed-model-output error before a clean success. Backend RBAC enforcement is real and pervasive. Both previously-reported critical fixes (run-level ERROR handling, self-contained backend tests) are confirmed still fixed, not regressed, under fresh, independent re-verification in this same session.

However, this audit also found genuine, currently-shipping problems that predate Phase 1–6 (inherited from earlier "Google-Studio-generated" UI work) but sit squarely inside the audit's own scope — the Test Case Generator and Settings pages:

- **One CRITICAL finding**: the "Application Source" connection check in the Test Case Generator fabricates a `CONNECTED` status with a fake HTTP 200/SSL-valid/ping-time (`Math.random()`-based) for *any syntactically valid URL*, with no real network call at all.
- **Two HIGH findings**: the Test Case Generator loads with hardcoded, pre-seeded "generated" test cases visible and saveable *before any real generation ever runs*; and the Settings page's non-AI tabs (Organization, Auth policy, Execution, Environments, Quality Policies, Notifications) accept and "successfully save" input that is never sent to any backend and is lost on reload.
- **Phase 5** (a dedicated RBAC Testing UI) was never built — confirmed by direct inspection, not assumed. This is honestly disclosed in the current code (`RBACTestingPage.tsx` shows a truthful "deferred" state), so it is a **known, disclosed gap**, not a fake capability — but it is still a missing capability relative to the original 6-phase plan.

None of the above five items were introduced by this session's Phase 1–6 implementation work; all five predate it and were left in place by prior sessions. They are reported here because the audit's own scope includes the Test Case Generator and Settings, and because the audit brief explicitly forbids treating a fake/disconnected finding as out-of-scope merely because it wasn't the auditor's own prior work.

Overall verdict: CONDITIONAL GO. See §2, §28.

---

2. Overall System Verdict

CONDITIONAL GO.

The core, audited Phase 1–6 execution chain (model discovery -> routing -> AI generation -> persistence -> execution handoff -> executor dispatch -> results -> evidence, plus the dedicated API Testing and Performance Testing surfaces) is genuinely real, connected, and now regression-tested to be self-contained. It is not a NO GO, because no part of *that* chain is fake, and every previously-reported critical defect is independently confirmed fixed in this audit, not merely re-asserted. It is not an unconditional GO, because real, currently-shipping fake/disconnected production behavior exists elsewhere in the same audited surface (Test Case Generator, Settings) and one planned capability (dedicated RBAC Testing UI) was never built.

---

3. Audit Scope

Per the brief: the full chain from AIQA startup through model discovery, routing, generation, persistence, execution handoff, every executor category (Functional/Browser, API, RBAC, Performance × Load/Stress/Spike/Volume), results/metrics/evidence, frontend, and report/audit-readiness. Backend: `services/orchestrator/src/*.rs`. Frontend: `forentend/src/**`. This is an audit-only pass; no features were built and no refactors were made during this task, per explicit instruction.

4. Repository Baseline

```
$ git status --porcelain
?? executors/playwright/tests/generated/     (pre-existing, gitignored runtime artifact)

$ git log --oneline -3
1670eed Add real Performance Testing (Phase 6) and two reliability fixes
18f4064 Add real AI test generation, execution compiler, Custom Testing, and API Testing (Phases 1-4)
0ab75de Wave 3 E2E execution, ACC-022 re-verification, and multi-unit backlog work (22-23 Aug 2026)

Remote: origin/claude/run-this-9eeb2e is up to date with local (1670eed), pushed.
```

Clean working tree, fully pushed. This audit was run against exactly the code at `1670eed`.

5. Architecture Overview

- **Backend**: Rust/Axum orchestrator (`services/orchestrator`), SQLite persistence (`store.rs`), bearer-key auth with SHA-256-hashed keys and 3-role RBAC (`auth.rs`), a policy/approval gate (`policy.rs`), a closed-vocabulary execution-plan compiler (`execution_plan.rs`) dispatching to a registry of real executors (`executor.rs`: Playwright/Newman/Pytest/Maestro/Schemathesis subprocess adapters + a native `CompiledApiExecutor`), a central AI provider/model router (`ai_router.rs`) with one genuinely live provider (Ollama) and three honestly-disclosed credential-presence-only stubs (Gemini/OpenAI/Anthropic), a real workload engine for performance testing (`performance_testing.rs`) with real Tokio task supervision, and a real evidence store (`evidence.rs`).
- **Frontend**: React/Vite (`forentend/`, the real, served frontend directory — the name is an intentional, pre-existing misspelling, not a typo), a typed API client (`aiqaApi.ts`), realtime updates via WebSocket (`RealtimeContext.tsx`), and per-phase pages (`TestCaseGeneratorPage.tsx`, `CustomTestingPage.tsx`, `APITestingPage.tsx`, `RBACTestingPage.tsx`, `PerformanceTestingPage.tsx`, `SettingsPage.tsx`, etc.).

6. End-to-End Data Flow

Stage	Input	Output	Storage/State	Consumer	Real Connection?
Model discovery	Env vars (`OLLAMA_BASE_URL`, `*_API_KEY`)	`AiProviderInfo[]`	In-memory, TTL-refreshed	Router, Settings UI	YES (Ollama live; others honestly stubbed — see §13)
Model routing	`AiTask`, provider list, `AiSettings`	Selected `(provider, model)` or `RoutingError`	In-memory	`generate()`	YES
Skill selection	User-selected built-in/custom skills	`SkillInput[]` in the generation request body	Session-only (custom skills), sent per-request	AI prompt construction	YES
AI generation	`requirement_text` + skills + provider/model	`GeneratedTestCaseDraft[]` or a real failure	None until saved	Test Case Generator UI	YES (live-verified, §20 Scenario A)
Generated cases	AI draft or manual entry	Persisted `TestCaseRow` (once saved)	SQLite `test_cases`	Test Case Generator, execution handoff	PARTIAL — see §7/§24 Finding 3 for the pre-generation default-seed caveat
Execution handoff	An `APPROVED` test case + executor choice	A real `RunPlan`/`TaskDef`	SQLite `runs`/`tasks`	Executor registry	YES (live-verified, §20 Scenario B)
API execution	Method/URL/headers /body/assertions	Real HTTP response + assertion results	Evidence store + `runs`/`tasks`	API Testing page	YES (live-verified, §20 Scenario C)
RBAC execution	Role + resource + action (via any real endpoint)	Real 200/403	Audit log	Any authenticated caller	YES for enforcement; NO dedicated testing UI (§11)
Performance execution	Schedule (Load/Stress/Spike/Volume) + target	Real samples + metrics + verdict	`performance_runs` + evidence	Performance Testing page	YES (live-verified, §20 Scenario E)
Results	Task/run/performance-run outcome	Status + reason + metrics	SQLite	Execution Hub, Performance Testing page	YES
Evidence	Raw log/sample content	Object on disk + DB row	`LocalEvidenceStore` + `evidence` table	Evidence Viewer	YES
Reports	Findings/failure clusters/test intelligence	Aggregated report data	SQLite (derived)	Reports page	YES (pre-existing, unaffected by this session; not independently re-audited here — out of the Phase 1–6 change scope)

7. Phase 1 Review — AI Integration

Implemented

Real generation request path: `TestCaseGeneratorPage.tsx` -> `aiqaApi.testCases.generateAi()` -> `POST /projects/:id/test-cases/generate-ai` (`api.rs`) -> `ai_generation::generate_test_cases` -> `AiInfrastructure::generate()` (`ai_router.rs`) -> real Ollama HTTP call -> real structured-JSON parse -> `GeneratedTestCaseDraft[]`. `generation_method` (`"ai"` vs `"fallback"`) and `provider`/`model` fields make the source of a result explicit and non-fakeable from the frontend. Skills are genuinely included: selected skill IDs/descriptions are serialized into the request body and resolved into real prompt instructions server-side (`resolve_skill_instructions`).

Partial

Provider/model discovery is real only for Ollama; Gemini/OpenAI/Anthropic are honestly-disclosed, credential-presence-only stubs with `reachable: false` always and zero real models — this is a real, but explicitly and accurately disclosed, architectural gap (not a fake "available" claim; see §13).

Missing

Nothing "missing" beyond the disclosed Gemini/OpenAI/Anthropic gap above — there is no expectation in the brief that those be live-integrated in this phase.

Evidence

- Live (this audit, §20 Scenario A): a genuine `TIMEOUT` fallback, a genuine `"invalid structured output: missing field \expected_result\"` fallback, and a genuine success (`generation_method: "ai"`, `provider: "ollama"`, `model: "gemma3:12b"`, real, on-topic generated content) — three different real outcomes, never masked into a fake uniform success.
- Code: `ai_router.rs`'s `route_among`/`generate` (real routing, real retry-with-fallback only in AUTO mode, real honest failure otherwise) — independently confirmed by the dedicated research agent in this audit with file:line citations.
- Trace-the-hops table:

Hop	Status
Frontend component -> frontend API service	CONNECTED
Frontend API service -> backend route	CONNECTED
Backend route -> service/orchestrator	CONNECTED
Orchestrator -> model router	CONNECTED
Model router -> provider/model	CONNECTED (Ollama); PARTIALLY CONNECTED (Gemini/OpenAI/Anthropic — configured-check only, never a live model call, honestly disclosed)
Provider -> response -> generated test cases	CONNECTED

Verdict

CONNECTED, real, and truthful. No fake "AI generated successfully" state exists in the generation call path itself (see §24 Finding 3 for a *different*, pre-generation UI-default issue that is not part of this call path).

8. Phase 2 Review — Execution Handoff

Implemented

Approval gate is server-enforced (`api.rs`, a `TestCaseStatus::Approved` check returns 409 otherwise, independent of any frontend restriction). `compile_and_execute` runs the real, unmodified deterministic/AI compiler (`execution_plan.rs`) and real SSRF-guarded `validate_plan`. The resulting plan is materialized into a real `TaskDef` and dispatched through the exact same `create_run_core`/`ExecutorRegistry` path the manual Execution Hub uses — no second execution lifecycle. An unregistered executor name produces a real `NO_COMPATIBLE_EXECUTOR` `InfraError`, never a silent fallback to a simulated pass (the Phase 2.1 fix, confirmed still in place by direct code read: `executor.rs`'s `ExecutorRegistry::resolve` only falls back to `SimulatedExecutor` when no executor name is given at all — an explicit, invalid name is a real error). Evidence is written to real disk (`std::fs::write`). The frontend learns of results via a real WebSocket subscription (`RealtimeContext`), not polling or an optimistic update. Traceability (`mark_test_case_executed`) links the originating test case to the real `run_id`/`task_id`.

Partial

If the traceability UPDATE's `WHERE status = 'APPROVED'` guard doesn't match (a real race, not expected in normal use), the link-back degrades to a logged warning rather than a hard error — the underlying execution itself is unaffected and still real.

Missing

Nothing found.

Evidence

- Live (this audit, §20 Scenario B): saved a real test case -> validated -> approved -> previewed a real `READY` deterministic compile -> handed off with `compile_and_execute: true` -> real run reached `state: "COMPLETED"`, task `"PASSED"` -> re-read the test case and confirmed `status: "EXECUTED"`, `executed_run_id`/`executed_task_id` both set to the real run/task IDs.
- Code: dedicated research agent traced every hop with file:line citations (`api.rs:714-781` handoff handler, `execution_plan.rs:360-422` compile/validate/materialize, `engine.rs`/`executor.rs:63-68` real dispatch and the non-fallback error path, `evidence.rs:24-43` real disk writes, `store.rs:969-983` traceability UPDATE).

Verdict

CONNECTED, genuinely, end-to-end, confirmed by both independent code tracing and fresh live execution in this audit.

9. Phase 3 Review — Custom Testing

Implemented

`routeConfig.tsx:246-256` — the `/custom-testing` route points at a real `CustomTestingPage` with `designStatus: 'new-follows-approved-language'` (not a placeholder bucket), and its own comment confirms it moved out of `unsupported-not-implemented` once `custom_testing.rs`'s real backend shipped. The backend module reuses two existing, real primitives rather than inventing a second generation path: `AiTask::RequirementAnalysis` (a real router call, previously unused in the taxonomy until this phase wired it) for the "strategy" half, and the exact, unmodified `ai_generation::generate_test_cases` for the "cases" half. Module/project selection is real (uses the actual selected project from context); "scope" is honestly labeled as user-provided free text (the `Project` entity has no module metadata field to select from — a disclosed, not fabricated, limitation).

Partial

None beyond the same Gemini/OpenAI/Anthropic discovery gap already disclosed in §7/§13, since this phase's AI call goes through the same router.

Missing

Nothing found relative to the phase's own stated scope.

Evidence

- Code: `services/orchestrator/src/custom_testing.rs` present, `routeConfig.tsx:246-256` confirmed real component/designStatus (checked directly in this audit, not assumed from a prior report).
- This phase reuses the exact same `ai_router`/`ai_generation` call path independently re-verified live in this audit (§20 Scenario A) — no separate live re-run was performed for Custom Testing specifically, since it would exercise identical underlying code already proven live in this session.

Verdict

CONNECTED. Genuinely implemented, not decorative — confirmed by direct, current code inspection in this audit.

10. Phase 4 Review — API Testing

Implemented

Real request construction (method/URL/query/headers/body), real `reqwest` execution via `CompiledApiExecutor`/`execute_plan_once` (no mocked executor), real per-request timing, real header/body capture with sensitive-header redaction, real per-assertion evaluation, real SSRF guard reuse, real evidence persistence, real frontend rendering of the actual response. "Execute" and "Save as Test Case" remain two genuinely distinct actions.

Partial

None found.

Missing

Nothing found relative to the phase's own stated scope.

Evidence

- Live (this audit, §20 Scenario C): a real GET to the orchestrator's own `/health` returned `status: 200`, `request_duration_ms: 8`, real response headers, `response_body_preview: "ok"`, and a real, passing assertion result.
- `cargo test` **self-containment, independently re-run in this audit (not trusted from the old report):** `cargo test` from a verified-clean environment (`netstat` confirmed no listener on 8080 immediately before the run) -> `298 passed; 0 failed; 2 ignored`. The 2 ignored are the pre-existing, legitimately-documented, unrelated Ollama-dependent tests. No manually-started server was used at any point in this run.
- SSRF preservation: the dedicated research agent independently confirmed `executor.rs`'s `is_blocked_ip`/`validate_target_url`/`ORCHESTRATOR_PORT` are unmodified and still reject RFC1918/link-local/cloud-metadata/non-documented-loopback targets, with the exact same passing unit tests as before.
- No new tests were ignored (confirmed: exactly the same 2 pre-existing `#[ignore]`s exist crate-wide, both Ollama-related, both unrelated to this fix).

Verdict

CONNECTED, and the previously-reported "Main Issue 2" (self-contained backend tests) is **CONFIRMED FIXED**, independently re-verified in this audit, not merely re-asserted from history.

11. Phase 5 Review — RBAC Testing

Implemented

RBAC ENFORCEMENT (a distinct, real capability): `auth.rs` implements a genuine 3-role (`Viewer < Operator < Admin`) system with SHA-256-hashed bearer keys, a real Axum auth middleware, `require_role`/`require_project_access` gates called at roughly 90 sites across `api.rs` covering every mutating endpoint from every phase (execution create/approve/reject/cancel, findings review, API-key admin, AI generation, custom testing, API testing, performance testing, project deletion), with read-only endpoints intentionally left open to any authenticated role (confirmed intentional by a passing test asserting a Viewer gets 200 on read but 403 on write for the same resource).

Partial

Nothing partial about the enforcement itself.

Missing

DEDICATED RBAC TESTING UI (a distinct, separate capability from the above) was never built. `RBACTestingPage.tsx` renders a static, honestly-labeled "DEFERRED — REAL FUTURE SCOPE" empty state, makes no backend calls, and its own doc comment states plainly that no role-permission-matrix entity or evaluation endpoint exists in the backend. No route path containing "rbac" exists anywhere in `services/orchestrator/src/`. No `AIQA_PHASE_5*` report file exists in the repository root. This is consistent with this session's own history: the Phase 5 prompt was sent truncated mid-transmission and never completed/resent, so Phase 5 was never started, let alone finished — this audit confirms that state is still accurate in the current code, not merely recalled from memory.

Evidence

- Live (this audit, §20 Scenario D): a real Viewer-role key got a real `403 FORBIDDEN` (`"role 'viewer' cannot perform this action (requires 'operator')"`) attempting a write, and a real `200` on a read of the same project; a real Operator-role key got a real `200` on the same write.
- Code: dedicated research agent's file:line citations for `auth.rs` (full file read), the ~90 `require_role`/`require_project_access` call sites in `api.rs`, `RBACTestingPage.tsx` (full file read, quoted), `routeConfig.tsx:198-206` (`designStatus: 'unsupported-not-implemented'`), and a `grep -i rbac` across the whole backend source finding zero dedicated routes.
- Independent audit-log corroboration: the real, persisted audit log recorded the live Scenario D actions with real outcomes.

Verdict

RBAC ENFORCEMENT: COMPLETE and CONNECTED. DEDICATED RBAC TESTING UI: MISSING, but honestly disclosed, not faked. These are two different capabilities and must not be conflated — the system's real security posture does not depend on the missing UI.

12. Phase 6 Review — Performance Testing

Implemented

Real Load/Stress/Spike/Volume workloads, each with genuinely distinct real scheduling behavior; real per-request sampling via the same core `execute_plan_once` the API Testing page uses; real, documented, unit-tested percentile/metrics computation; real cancellation (races and interrupts an in-flight request, not just "stop scheduling"); server-enforced safety limits; real, immediate run-level `ERROR` detection via genuine Tokio `JoinHandle` supervision (the previously-reported "Main Issue 1" fix); a DB-level terminal-state idempotency guard so a row can never flip from `ERROR` back to `COMPLETED`; startup recovery still correctly produces the distinct `INTERRUPTED` status for a genuinely orphaned row after a real process crash.

Partial

None found beyond what is already honestly disclosed in the existing Phase 6 reports (e.g., a full *process* crash — as opposed to an in-process task panic — is, by design, only recoverable via the pre-existing `INTERRUPTED` sweep, not the new `ERROR` supervisor; this is correct and intentional, not a gap).

Missing

Nothing found relative to the phase's own stated scope.

Evidence — Main Issue 1 re-verification (independently re-run in this audit)

- Live: a small real Load workload with `max_p95_ms: 1000` finished `status: COMPLETED`, `verdict: PASSED`, `error_reason: null`, `sample_count: 8927` — normal execution and threshold evaluation both confirmed still correct post-fix.
- Live: a real cancellation of a 15-second Load run stopped the run in `284ms` with `status: CANCELLED`, `verdict: CANCELLED` — cancellation confirmed still correct, never misclassified as `ERROR`.
- Code (not re-derived from scratch, but independently re-read in this audit): `performance_testing.rs`'s `supervise_workload_task`/`SupervisionFailure` genuinely owns the workload's `JoinHandle` and converts a real panic into an immediate, persisted `ERROR` with a safe reason, real partial metrics, and a frozen `finished_at`; `store.rs`'s `finalize_performance_run` now guards its `UPDATE` with `WHERE status = 'RUNNING'`, confirmed present in the current file.
- Test suite: the 5 dedicated `phase6_error_*` integration tests and the `performance_testing::tests` module's 30 tests are part of the same `298 passed, 0 failed` full-suite result this audit independently re-ran (§10, §19).

Verdict

CONNECTED, and the previously-reported "Main Issue 1" (run-level ERROR handling) is **CONFIRMED FIXED**, independently re-verified live in this audit.

13. Startup Model/API Discovery Review

Item	Status
Providers in code	Ollama (local), Gemini, OpenAI, Anthropic (all in `ai_router.rs`)
Credential configuration	Env vars (`OLLAMA_BASE_URL` optional, `GEMINI_API_KEY`/`OPENAI_API_KEY`/`ANTHROPIC_API_KEY`)
Availability genuinely checked?	**Ollama: YES** — a real `GET {base_url}/api/tags` HTTP call, parsed, with genuine `Healthy`/`Unavailable`/`Error` outcomes. **Gemini/OpenAI/Anthropic: NO** — presence of a non-empty env var alone sets `configured: true`, `reachable` stays `false` unconditionally; no HTTP call is ever made. This is explicitly disclosed in the module's own doc comment and unit-tested to confirm `reachable` never becomes `true` this way.
Model discovery automatic?	Yes for Ollama — a real background task at startup (`main.rs`, non-blocking, `tokio::spawn`) plus TTL-based re-discovery (`ensure_fresh`) on each real use.
Selection based on real availability?	Yes — `score_model`/`route_among` operate only over genuinely discovered, capability-scored candidates; a stub provider with zero real models can never be selected to actually generate anything (confirmed: even if "configured," no `AiExecutor` is registered for Gemini/OpenAI/Anthropic in `from_env()`, so `generate()` would hit a real, honest "no executor" failure rather than a fabricated call).
Preferred provider unavailable?	A real `RoutingError` (`ProviderUnavailable`/`ModelUnavailable`/`NoCompatibleModel`) in MANUAL mode; a real, executed fallback attempt against other genuine candidates in AUTO mode with `fallback_enabled: true`.
All providers unavailable?	A real `GenerationOutcome::Failed { reason }` — surfaced to the frontend as a distinct `generation_method: "fallback"`/`fallback_reason` (Test Case Generator) or `generation_source: "UNAVAILABLE"` (Custom Testing) — never a fabricated success. Live-verified in this audit's own real timeout/parse-failure attempts (§20 Scenario A).
Settings reflects actual configuration?	Yes — `GET /api/v1/ai/status`/`/ai/settings` return only the real, current discovery snapshot and persisted settings; the frontend's `SettingsPage.tsx` AI panel renders directly from these, with no hardcoded provider-status strings.
Secrets exposed to frontend?	No — `AiSettings`/`AiProviderInfo` structs carry no credential field at all; confirmed by dedicated backend tests asserting a credential value never appears in any serialized provider-info response.
Fake "Connected" status?	No, for AI provider status specifically (see §24 Finding 1 for a *different*, unrelated fake "Connected" status in the Test Case Generator's Application Source panel — not part of AI provider discovery).

Classification: ACTUALLY IMPLEMENTED NOW for Ollama discovery/routing/generation; **PARTIALLY IMPLEMENTED**, honestly, for Gemini/OpenAI/Anthropic (configuration-presence detection only, clearly and consistently disclosed as such everywhere it's surfaced — not claimed as "automatic discovery").

14. Settings Truthfulness Review

Setting	UI Exists	Saves	Persists	Used at Runtime	Truthful Status
AI provider status display	Yes	N/A (read-only)	N/A	Yes	TRUTHFUL — real `GET /ai/status`
"Scan Providers"	Yes	N/A (triggers refresh)	N/A	Yes	TRUTHFUL — real `POST /ai/refresh`
AI mode/preferred provider/model/fallback ("Save AI Settings")	Yes	Yes	Yes (SQLite `ai_settings`, real `PUT /ai/settings`)	Yes (updates the live in-memory router immediately)	TRUTHFUL
Organization Name / Project Key / Data Retention / Timezone ("Save Settings"/"Save Changes")	Yes	UI-only (local React state)	**NO**	**NO**	**NOT TRUTHFUL** — shows a "Configuration Saved" toast with no backing endpoint; see §24 Finding 4
Authentication & SSO toggle, "Control Plane Daemon API Token"	Yes	UI-only	**NO**	**NO**	**NOT TRUTHFUL** — a fabricated "Status: ACTIVE • Expires in 180 days" and a dead "Rotate Token" button; see §24 Finding 5
Execution/Runner pool, Target Environments, Quality Policies & Gates, Alerts & Notifications tabs	Yes	UI-only	**NO**	**NO**	**NOT TRUTHFUL** — same disconnected `handleSave`/`updateSettings` local-state-only path as above

No credential value is ever exposed improperly (§13). The disconnected tabs above do not accept or persist any credential — they are governance/policy-style settings, not secrets — so this is a truthfulness/trust problem, not a secret-exposure problem.

15. Persistence Matrix

Entity	Status
Selected model/provider preference	PERSISTED (SQLite `ai_settings`)
Provider configuration (credentials)	NOT IMPLEMENTED as app-managed persistence — read from process env vars only, by design (secrets are never written to AIQA's own store)
Selected skills (built-in)	SESSION ONLY — sent per-request, not itself a saved setting
Custom skills	SESSION ONLY, honestly disclosed to the user in the UI itself ("will exist only for this browser session")
Generated test cases	SESSION ONLY until explicitly saved; PERSISTED (SQLite `test_cases`) once saved — see §24 Finding 3 for the pre-generation default-seed caveat
Execution runs	PERSISTED (SQLite `runs`/`tasks`)
API execution results (ad-hoc)	PERSISTED via the same evidence/run/task mechanism every execution uses (raw log + structured `RESULT_SUMMARY_JSON` line)
RBAC evidence/results	NOT IMPLEMENTED — no dedicated entity exists (consistent with §11's finding that the dedicated feature itself was never built); the general, real Audit Trail is the closest persisted evidence of enforcement
Performance runs	PERSISTED (SQLite `performance_runs`)
Performance metrics	PERSISTED (`metrics_json` column; raw per-request samples via the evidence store)
ERROR reason	PERSISTED (`error_reason` column, both for genuine task-panic `ERROR` and for startup-recovery `INTERRUPTED`)
Reports/evidence	PERSISTED (`evidence` table + real files on disk via `LocalEvidenceStore`)

16. Frontend Status Truthfulness Matrix

Status	Origin	Backend-derived / local / static	Can be falsely displayed?	Unavailable functionality disclosed?
AI Connected / provider status	`GET /ai/status`	Backend-derived	No	Yes (`NOT_CONFIGURED`/ `Unavailable` shown truthfully)
Model Available	Same	Backend-derived	No	Yes
Generating	In-flight `fetch` promise state	Local, but tied to a real request	No	N/A
Generated	`generation_method`/`provider`/`model` in the real response	Backend-derived	No, for the actual generation call	See Finding 3 — the page's *pre-generation default state* is not backend-derived and can look identical to a real result
Saved	Real `POST /test-cases` response (a real persisted ID)	Backend-derived	No	N/A
Execution Started / Running / PASS / FAIL / ERROR / CANCELLED / INTERRUPTED	Real run/task/performance-run status fields	Backend-derived	No	Yes (each is a distinct, real backend enum value; live-verified in §20)
"Connected" (Application Source panel)	A local `setTimeout` + `Math.random()`	**Local/static, not backend-derived**	**YES — always shows Connected for any syntactically valid URL**	**No — not disclosed to the end user anywhere in the UI**

17. Security Review

Finding	Severity
SSRF guard (`is_blocked_ip`/`validate_target_url`/`ORCHESTRATOR_PORT`) unmodified, still blocks RFC1918/link-local/cloud-metadata/non-documented loopback; verified via the existing, passing test set	INFO (confirmed intact, no finding)
API keys hashed (SHA-256) before storage; raw key never persisted, returned exactly once at mint time	INFO (confirmed correct)
No credential field in any `AiSettings`/`AiProviderInfo` serialization path; backed by dedicated tests	INFO (confirmed correct)
No raw API key/secret found in any `tracing::` log call across `ai_router.rs`/`auth.rs`/`api.rs`	INFO (confirmed correct)
The test-only, self-contained `:8080` HTTP fixture added for backend test self-containment is compiled only under `#[cfg(test)]` and does not touch or weaken the production SSRF guard	INFO (confirmed correct; this was the explicit target of the prior fix's own security review, re-confirmed here)
Application Source panel fabricates a "connected"/"SSL valid" status without ever making a real request	LOW (as a security issue specifically — it does not itself expose or transmit anything unsafely; its severity as a *truthfulness* issue is CRITICAL, tracked in §24 Finding 1, not duplicated here as a security defect)

No CRITICAL, HIGH, or MEDIUM **security** findings in the Phase 1–6 scope. The one CRITICAL finding of this audit (§24 Finding 1) is a truthfulness/deception issue, not a security vulnerability — it does not leak data, bypass auth, or create an injection/SSRF vector.

18. Fake/Stub/Placeholder Audit

A full keyword sweep (`fake, mock, demo, sample, placeholder, stub, dummy, simulated, hardcoded success, hardcoded connected, hardcoded result, Math.random, TODO, FIXME, not implemented, setTimeout, fake upload, fake execution, fake save, fake connection, seeded test case`) was run across all `services/orchestrator/src/*.rs` and `forentend/src/**/*.{ts,tsx}` files by a dedicated research pass, with every matching file reviewed.

Backend Rust code (Phase 1–6): clean. Every match is either a `#[cfg(test)]`-only fixture (Category A) or an honestly-disclosed architectural gap already documented in its own doc comment (Category C) — including the Gemini/OpenAI/Anthropic discovery stub, re-confirmed still accurately disclosed.

Frontend: 5 real findings, detailed in §24, all Category **D/E/F** (temporary/fake/disconnected), all pre-existing (not introduced by this session's Phase 1–6 work), all inside the Test Case Generator or Settings pages. Every other match (routine `*.test.tsx` mocked `fetch`, genuine HTML `placeholder="..."` attributes, legitimate debounce `setTimeout`s, the honestly-disclosed "Case C" not-implemented pages) was reviewed and classified as A/B/C — no further action needed there.

19. Test Results

Backend

```
$ netstat -ano | grep ":8080"          # -> no output; environment confirmed clean
$ cargo test
test result: ok. 298 passed; 0 failed; 2 ignored; 0 measured; 0 filtered out; finished in 10.37s
test result: ok. 0 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.00s   (main.rs bin)
test result: ok. 0 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.00s   (doc-tests)
```

The 2 ignored are the pre-existing, legitimately-documented (`#[ignore = "requires a real, running local Ollama server..."]`) tests — unrelated to this session, unaffected, and this audit additionally proved a real Ollama round trip live (§20 Scenario A), independent of those specific tests.

Frontend

```
$ npx vitest run
Test Files  22 passed (22)
Tests       401 passed (401)
```

Typecheck

```
$ npx tsc --noEmit -p .
(no output — exit code 0)
```

Production Build

```
$ npm run build
✓ built in 25.22s
(exit code 0; one pre-existing chunk-size advisory warning, unrelated to correctness)
```

No new failures were caused by this audit (it made no code changes). No test was newly ignored or skipped.

20. Live Verification

- **Scenario A — AI Generation (LIVE VERIFIED).** Started the real orchestrator, started the real local Ollama service (previously not running), forced a real discovery refresh (`POST /ai/refresh` -> real `HEALTHY`, real `gemma3:12b` model list). Issued three real generation requests against the real provider: (1) a genuine `TIMEOUT` fallback under the default timeout budget, (2) a genuine `"invalid structured output: missing field \expected_result\"` fallback after extending the timeout, (3) a genuine success — `generation_method: "ai"`, `provider: "ollama"`, `model: "gemma3:12b"`, two real, on-topic generated test cases. All three outcomes were truthfully, distinctly reported; none was masked as a different outcome.

- **Scenario B — Generated Case Execution (LIVE VERIFIED).** Saved a real test case, validated it, approved it, previewed a real `READY` deterministic compile, handed it off with `compile_and_execute: true`. The real run reached `state: "COMPLETED"` with the task `"PASSED"`, and the originating test case's own record was confirmed updated to `status: "EXECUTED"` with real `executed_run_id`/`executed_task_id` values matching the real run.

- **Scenario C — API Execution (LIVE VERIFIED).** A real GET to the orchestrator's own `/health` (the one documented, SSRF-allowed self-target) returned a real `200`, real `8ms` measured latency, real response headers, `response_body_preview: "ok"`, and a real, passing `ASSERT_STATUS 200` result.

- **Scenario D — RBAC (LIVE VERIFIED).** A real Viewer-role key received a real `403 FORBIDDEN` attempting a write and a real `200` on a read of the same resource; a real Operator-role key received a real `200` on the same write. All three actions independently corroborated in the real, persisted audit log.

- **Scenario E — Performance (LIVE VERIFIED).** A real small Load workload with a configured threshold finished `COMPLETED`/`PASSED` with `8927` real samples and `error_reason: null`; a real 15-second Load run was cancelled via the real endpoint and genuinely stopped in `284ms`, reporting `CANCELLED`/`CANCELLED`. (A genuine, controlled internal-ERROR live-HTTP trigger was not attempted in this audit — it was already established in the prior ERROR-fix session, and re-attempting it live would require the same test-only supervision seam already covered by 5 passing integration tests re-run as part of §19's `298 passed` result; this is a **TEST VERIFIED**, not **LIVE VERIFIED**, claim for that one specific sub-case, disclosed here rather than re-asserted as live.)

- **Application Source "Connected" claim** — **CODE VERIFIED** (not live-UI-verified in this pass, since it would require opening a browser; the fabrication is unambiguous and fully provable from source alone — see §24 Finding 1).

Nothing in this audit was claimed as live-verified without an actual command/request having been run and its real output captured above.

21. Main Issues Previously Fixed

Issue 1 — Run-level ERROR handling

Current status: FIXED, independently re-verified in this audit — live re-run of normal execution (COMPLETED/PASSED) and cancellation (CANCELLED), plus a fresh full-suite run including all 5 `phase6_error_*` tests and the 3 supervision-specific unit tests, all passing (§10 test citation applies identically to the same suite run; see §19).

Issue 2 — Self-contained backend tests

Current status: FIXED, independently re-verified in this audit — `cargo test` run from a `netstat`-confirmed clean environment (no manual server) produced `298 passed; 0 failed; 2 ignored`, matching the fix's own reported result exactly, not merely re-quoted from the prior report.

22. Phase Acceptance Matrix

Phase	Requirement	Evidence	Status
1	Real AI integration	§7, §20 Scenario A	COMPLETE
1	Model router	§7, §13	COMPLETE
1	Truthful provider status	§13, §16	COMPLETE
1	Skills state	§7, §6	COMPLETE
2	Real execution handoff	§8, §20 Scenario B	COMPLETE
2	Result return path	§8	COMPLETE
3	Custom testing	§9	COMPLETE
4	Real API execution	§10, §20 Scenario C	COMPLETE
4	Self-contained tests	§10, §19, §21 Issue 2	COMPLETE
4	SSRF preserved	§10, §17	COMPLETE
5	Backend RBAC enforcement	§11, §20 Scenario D	COMPLETE
5	RBAC testing UI	§11	MISSING (honestly disclosed)
6	Load	§12, §20 Scenario E	COMPLETE
6	Stress	§12 (code + prior live evidence this session; not independently re-run live in this audit pass)	COMPLETE
6	Spike	§12 (same)	COMPLETE
6	Volume	§12 (same)	COMPLETE
6	Cancellation	§12, §20 Scenario E	COMPLETE
6	Run-level ERROR	§12, §21 Issue 1	COMPLETE
6	Partial metrics	§12	COMPLETE
6	Startup recovery	§12	COMPLETE
1–6	No critical fake production states	§18, §24	**PARTIAL — 1 CRITICAL finding outside the core chain, inside the audited Test Case Generator surface**
1–6	Backend regression clean	§19	COMPLETE
1–6	Frontend regression clean	§19	COMPLETE

23. Critical Gaps

Gap 1

Title: Application Source connection check fabricates a real-looking "Connected" status with no real network call **Phase:** Test Case Generator (upstream of Phase 1) **Severity:** CRITICAL **Exact file/module:** `forentend/src/components/test-generator/ApplicationSourceSection.tsx`, lines 186-205 **Current state:** `handleConnect` performs only client-side URL-syntax validation, then after a fixed 600ms `setTimeout`, unconditionally sets `status: 'CONNECTED'` with `details: { httpStatus: 200, sslValid: true, pingMs: Math.floor(25 + Math.random() * 45) }` — for any syntactically valid URL, including unreachable or nonexistent hosts. **Why incomplete:** No `fetch`/network call exists anywhere in the function. **User impact:** A user is shown a fabricated "SSL: Valid • HTTP 200 OK" and a fake, randomized ping time for a target that may not exist or be reachable at all, with no disclosure that this is not a real check. **Evidence:** Independently confirmed by direct source read in this audit (quoted above); originally surfaced by a dedicated research pass with the same file:line citation. **Recommended next action:** Either implement a real, safe connectivity check (e.g., a real backend-proxied HEAD/GET request, reusing the existing SSRF-guarded executor path) or remove the fabricated status entirely and honestly label the field as unverified until a real check exists.

Gap 2

Title: Document Upload panel fabricates a file-ingestion progress bar; file content is never actually read/transmitted **Phase:** Test Case Generator (upstream of Phase 1) **Severity:** HIGH **Exact file/module:** `forentend/src/components/test-generator/DocumentUploadSection.tsx`, lines 125-162 **Current state:** A `setInterval`-driven progress animation reaches "Ready" with no `fetch`/API call anywhere in the component; the file's bytes never leave the browser. **Why incomplete:** No backend endpoint for document ingestion is called by this component at all. **User impact:** A user reasonably believes their uploaded PRD/FRD/SRD document was read and will inform generation; it is not analyzed at all (the backend-facing request text does honestly say so, but that disclosure never reaches the human watching the fake progress bar). **Evidence:** Confirmed by dedicated research pass with file:line citation; not independently re-read line-by-line by the primary auditor in this pass, but the underlying claim (no fetch call in the component) is straightforward to re-confirm by inspection. **Recommended next action:** Either implement real document ingestion (upload + real text extraction) or replace the fake progress bar with an honest "not yet supported" state, consistent with how other genuinely-deferred features in this codebase are disclosed.

Gap 3

Title: Test Case Generator loads with hardcoded, pre-seeded "generated" test cases visible and saveable before any real generation **Phase:** Test Case Generator (upstream of Phase 1) **Severity:** HIGH **Exact file/module:** `forentend/src/context/AIQAContext.tsx`, lines 112-188 (default `activeGenerationSession` state, `currentStep: 'GENERATED_CASES'`) **Current state:** Independently confirmed in this audit: `activeGenerationSession` is initialized (not `null`) with `currentStep: 'GENERATED_CASES'` and a list of fully-written, plausible-looking fake test cases (e.g., an OAuth PKCE case) on first component mount, before a user has ever clicked "Generate." **Why incomplete:** The component's own comment (line 121-123) describes this as pre-existing, previously-disclosed "sample dashboard content" deliberately left in place. **User impact:** A user landing on the page sees what looks exactly like a completed, real AI generation result and can click "Save to Project" (`canSave` is true whenever `generatedCases.length > 0`, with no distinction for this default-seeded case), persisting fabricated content into the real backend as if it were genuine AI output. **Evidence:** Confirmed directly by the primary auditor in this pass (`Read` on `AIQAContext.tsx:108-137`). **Recommended next action:** Default `activeGenerationSession` to `null` (or an explicit `currentStep: 'IDLE'`/`'REQUIREMENT_INPUT'`) so the Generated Cases view is only ever reachable after a real generation call, exactly as was already done for the `requirementText` field in the same object (per its own adjacent comment).

Gap 4

Title: Most Settings tabs accept and "successfully save" input with no backend persistence **Phase:** Settings (cross-cutting, not itself a Phase 1–6 deliverable, but touched by the Phase 1–6 AI panel that sits alongside it) **Severity:** HIGH **Exact file/module:** `forentend/src/components/pages/SettingsPage.tsx`, `handleSave` (lines ~415-425) and

every non-AI tab (General, Authentication non-AI parts, Execution, Environments, Quality Policies, Notifications) **Current state:** `handleSave` calls only `updateSettings(formState)`, a local `AIQAContext` state setter with no network call; a "Configuration Saved" toast is shown regardless. **Why incomplete:** No backend endpoint exists for any of these settings categories. **User impact:** A user configuring organization details, SSO, runner concurrency, environment URLs, pass-rate policies, or notification channels sees a convincing success confirmation, but a page refresh silently reverts everything. **Evidence:** Confirmed by dedicated research pass with file:line citations; the AI-specific tab's own real save path (§14) was separately, independently confirmed correct by the primary auditor. **Recommended next action:** Either wire these tabs to real backend persistence or add an explicit, visible "not yet persisted" disclosure, matching the honesty standard already applied elsewhere in this codebase (e.g., the custom-skill session-only disclosure).

Gap 5

Title: Fabricated "Control Plane Daemon API Token" status and a non-functional "Rotate Token" control **Phase:** Settings **Severity:** MEDIUM **Exact file/module:** `forentend/src/components/pages/SettingsPage.tsx`, lines ~615-626 **Current state:** A static `"Status: ACTIVE • Expires in 180 days"` string and a "Rotate Token" button with no `onClick` handler. **Why incomplete:** No backing entity for this token exists. **User impact:** Low direct risk (nothing is actually rotated or trusted downstream), but it misrepresents a security-adjacent control as functional. **Evidence:** Confirmed by dedicated research pass with file:line citation. **Recommended next action:** Remove or honestly label as non-functional until a real token entity/rotation endpoint exists.

Gap 6

Title: Dedicated RBAC Testing UI was never built **Phase:** 5 **Severity:** MEDIUM (downgraded from what a "missing phase" might otherwise imply, specifically because it is honestly disclosed and does not misrepresent anything) **Exact file/module:** `forentend/src/components/pages/RBACTestingPage.tsx` **Current state:** A real, honest "DEFERRED" empty state with no backend calls; no backend endpoint exists to support one. **Why incomplete:** The Phase 5 prompt that would have specified this feature was sent truncated and never resent/completed in this session's history. **User impact:** No dedicated role/permission/resource/action ALLOW/DENY visualization exists; users must infer enforcement correctness from the general Audit Trail or from real functional-endpoint behavior. **Evidence:** §11. **Recommended next action:** Build Phase 5 properly once its full requirements are specified (see §26).

24. Remaining Fake/Disconnected Areas

Summarized from §23: Gaps 1, 2, 3, 4, and 5 are all currently-shipping, real, user-facing fake or disconnected behavior. Gap 1 is the single CRITICAL item. None of the five originate from this session's own Phase 1–6 implementation work (verified: none of the affected files — `ApplicationSourceSection.tsx`, `DocumentUploadSection.tsx`, `AIQAContext.tsx`'s seed data, `SettingsPage.tsx`'s non-AI tabs — appear in this session's `git diff`/commit history for Phases 1–6 or either reliability fix).

25. Known Limitations

- Phase 6's "genuine controlled internal ERROR" path was re-confirmed via the existing test suite in this audit, not via a fresh live-HTTP trigger (§20) — an intentional, previously-disclosed limitation (a real panic cannot be triggered through the live HTTP API without touching the untouched Load/Stress/Spike/Volume logic itself).
- Stress/Spike/Volume were not independently re-run live in this specific audit pass (Load and Cancellation were); their correctness rests on the extensive live verification already performed and documented in the original Phase 6 report plus the same, currently-passing automated test suite re-run in this audit (§19).
- The Reports/Findings/Test-Intelligence pipeline (downstream of "Results"/"Evidence" in §6) was not independently re-audited in this pass — it predates Phase 1–6 and was out of this session's change scope; its data flow is marked YES in §6 on the basis of it being pre-existing, previously-verified functionality, not fresh evidence gathered here.
- Gemini/OpenAI/Anthropic remain genuinely unintegrated beyond credential-presence detection — an accepted, disclosed architectural gap, not attempted to be closed in this audit (auditing only, no fixes).

26. Recommended Next Phase

Fix Gap 1 (the fabricated Application Source "Connected" status) first, followed immediately by Gaps 2–5 as a single "Test Case Generator & Settings truthfulness" cleanup pass.

Why this has priority over building Phase 5: Gap 1 is the one CRITICAL finding in this entire audit, it actively misrepresents real-looking network/security facts (HTTP status, SSL validity) to a user inside the exact same Test Case Generator flow this audit's own scope opens with, and — unlike the missing RBAC Testing UI (Gap 6), which is honestly disclosed and therefore lower-urgency by this codebase's own established truthfulness standard — Gap 1 is not disclosed to the user at all. Fixing fake truthfulness in an already-shipped, actively-used surface is higher-value and lower-risk than building an entirely new Phase 5 feature from an as-yet-unspecified brief. Phase 5 (dedicated RBAC Testing UI) should follow once Gaps 1–5 are closed and its own requirements are re-specified in full (the original brief for it was never completed).

27. Final Evidence Matrix

Capability	Code	Tests	Live	Persistence	UI	Verdict
AI generation (Ollama)	■	■	■	■ (once saved)	■	REAL
AI generation (Gemini/OpenAI/Anthropic)	■■ (stub, disclosed)	■ (stub behavior tested)	N/A (not configured)	N/A	■ (honest)	HONEST STUB
Execution handoff	■	■	■	■	■	REAL
API Testing	■	■	■	■	■	REAL
RBAC enforcement	■	■	■	■ (audit log)	N/A (no dedicated UI)	REAL
RBAC testing UI	N/A	N/A	N/A	N/A	■■ (honest placeholder)	MISSING (disclosed)
Performance Testing (Load/Stress/Spike/Volume)	■	■	■ (Load; others code+prior-session-live)	■	■	REAL
Run-level ERROR handling	■	■	■	■	■	REAL, FIXED
Backend test self-containment	■	■	N/A	N/A	N/A	REAL, FIXED
Application Source "Connected" check	■	N/A	N/A (fake)	N/A	■	FAKE — CRITICAL
Document Upload ingestion	■	N/A	N/A (fake)	N/A	■	FAKE — HIGH
Default pre-seeded "generated" cases	■	N/A	N/A	N/A (only if user saves it)	■	FAKE — HIGH
Settings (non-AI tabs) persistence	■	N/A	N/A	■	■	DISCONNECTED — HIGH
Settings API token status/rotate	■	N/A	N/A	N/A	■	FAKE/DEAD — MEDIUM

28. Final Verdict

CONDITIONAL GO.

The Phase 1–6 core critical paths audited here — model discovery/routing, AI generation, execution handoff, API execution, RBAC enforcement, and performance testing including both previously-reported critical fixes — are genuinely connected, real, and independently re-verified in this pass via code, an independently re-run full test suite (298/298 backend, 401/401 frontend, clean typecheck, clean build), and live runtime evidence across five real end-to-end scenarios. Neither previously-fixed issue has regressed.

This is not an unconditional GO because real, currently-shipping fake and disconnected production behavior remains in the same audited surface (the Test Case Generator's Application Source and Document Upload panels, its default pre-seeded generated-cases state, and most of the Settings page) — one of these (Gap 1) is CRITICAL by this audit's own classification — and because Phase 5's dedicated RBAC Testing UI was never built. None of these five items are hidden; all are disclosed here with exact file:line evidence and a recommended fix.

29. Final Questions

1. **Can AIQA genuinely generate test cases using its configured AI path?** Yes — live-verified in this audit with a real Ollama call producing `generation_method: "ai"`, real provider/model attribution, and real, on-topic content (§20 Scenario A).

2. **Is model selection/routing real or decorative?** Real — `route_among`/`score_model` operate over genuinely discovered candidates; confirmed by code and by two different genuine failure modes (timeout, malformed output) observed live before a genuine success (§7, §20).

3. **Are unavailable providers shown truthfully?** Yes — `NOT_CONFIGURED`/`Unavailable` statuses are real and never overridden into a fake "Connected" for AI providers specifically (§13, §16). (A different, unrelated panel — Application Source — does fake a "Connected" status; see Gap 1.)

4. **Do skills actually enter the generation flow where claimed?** Yes — selected skills are serialized into the real request body and resolved into real prompt instructions server-side (§7).

5. **Can generated cases genuinely reach execution?** Yes — live-verified end-to-end, including the traceability link-back (§8, §20 Scenario B).

6. **Which executors are genuinely connected?** `CompiledApiExecutor` (native, live-verified), and — per code inspection and the existing, passing test suite — `PlaywrightExecutor`/`NewmanExecutor`/`PytestExecutor`/`MaestroExecutor`/`SchemathesisExecutor` (subprocess-based, real dispatch confirmed by the unmodified `ExecutorRegistry::resolve` logic and its own tests; not independently re-run live in this specific audit pass since none of Phase 1–6's own changes touched them).

7. **Which executor/UI paths remain incomplete?** The dedicated RBAC Testing UI (Gap 6) has no backend counterpart at all; everything else audited is complete relative to its own stated scope.

8. **Does API execution perform real HTTP requests?** Yes — live-verified with real, measured latency and real response headers (§20 Scenario C).

9. **Does `cargo test` run without manually starting a server?** Yes — independently re-confirmed in this audit from a `netstat`-verified clean environment (§19).

10. **Is SSRF protection still intact?** Yes — confirmed unmodified by direct code inspection in this audit (§17).

11. **Is RBAC backend enforcement real?** Yes — live-verified with real 403/200 outcomes for the same resource under different roles (§20 Scenario D).

12. **Is dedicated RBAC testing UI fully real, partial, or missing?** Missing, honestly disclosed (§11).

13. **Are Load/Stress/Spike/Volume genuinely executable?** Yes — Load live-re-verified in this audit; all four confirmed by code and by the passing, unchanged test suite from the original Phase 6 work (§12, §19).

14. **Is run-level ERROR genuinely reachable?** Yes — confirmed by the passing `phase6_error_*` integration tests exercising the real `supervise_workload_task`/`finalize_performance_run_as_error` path end-to-end (§12, §19, §21).

15. **Can a workload panic leave RUNNING orphaned while the process survives?** No — the real `JoinHandle` supervisor catches it and finalizes `ERROR` immediately (§12).

16. **Are ERROR and INTERRUPTED distinct?** Yes — confirmed both by code (separate trigger paths) and live, in the original ERROR-fix session, via a genuine process kill (re-confirmed here by re-reading the same, unmodified code and re-running its test coverage; not re-triggered live again in this specific pass).

17. **Are partial performance metrics truthful?** Yes — computed only from real, actually-completed samples, confirmed by passing tests asserting exact partial counts (§12).

18. **Are important Settings controls genuinely saved and used?** Only the AI Model/Provider settings — yes, genuinely (§14). Every other Settings tab is not (§14, Gap 4).

19. **What remains session-only?** Custom skills (honestly disclosed), unsaved generated test cases, and ad-hoc API-testing configuration state before "Save as Test Case" (§15).

20. **What remains local-UI-only?** Most of the Settings page (Gap 4), the Application Source "Connected" status (Gap 1), and the Document Upload progress bar (Gap 2).

21. **What fake/disconnected production behavior remains?** Five items, detailed in §23/§24 — one CRITICAL (Gap 1), three HIGH (Gaps 2-4), one MEDIUM (Gap 5).

22. **What critical gap remains, if any?** Gap 1 — the fabricated Application Source connection check.

23. **What should be implemented next?** Fix Gaps 1–5 (Test Case Generator and Settings truthfulness) before starting a new Phase 5 build (§26).

24. **Is AIQA ready to move beyond Phase 6?** Conditionally — the Phase 1–6 core chain itself is ready and regression-safe, but the truthfulness gaps in Gaps 1–5 should be closed first, since they sit inside the same Test Case Generator surface this audit's own scope treats as the system's front door.

End of report.